AI CODE AGENT PLATFORM

Complete Multi-Agent Architecture Blueprint

Python-First · 11 Specialized AI Agents · Full Agent Connection Map

FastAPI · CrewAI/AutoGen · Docker · PostgreSQL + pgvector · Redis · React + Monaco

Version 4.0	Status Living Document	Agents 11 Defined	Classification Confidential

TABLE OF CONTENTS

1. Objective & Vision

The AI Code Agent Platform is a fully autonomous, end-to-end software development system modeled on a human development agency. It is composed of 11 specialized AI agents working in coordination under a central Orchestrator — ingesting natural language requirements or PRD documents and delivering fully tested, deployed, production-ready applications with minimal human intervention.

1.1 Core Capabilities

- Requirement Ingestion — Parse natural language, Markdown, DOCX, or PDF requirement documents into structured machine-readable specs
- Multi-Agent Coordination — 11 specialized agents with clearly defined roles, triggers, inputs, and outputs
- Code Generation & Modification — Generate new files or precise unified diffs for existing repositories
- Secure Code Execution — Isolated Docker sandboxes with CPU, memory, network controls, and seccomp profiles
- Autonomous Debugging — Iterative error analysis, root cause identification, auto-fix loops (max 5 retries)
- Multi-Layer Validation — Syntax, type check, build, test, security scan, and LLM semantic correctness checks
- Automated Deployment — DevOps Agent generates Dockerfile, CI/CD pipelines, and deploys to cloud

1.2 Supported Workflows

Workflow	Input	Output	Agents Involved
New Project from PRD	PRD document / natural language spec	Full working codebase, deployed URL	Orchestrator → PM → Architect → Dev → QA → DevOps
Existing Repo Modification	Codebase + change request	Minimal diff / patch + PR	Orchestrator → Planner → RAG → Executor → Validator
Bug Fix	Error logs + failing code	Fixed code + regression test	Orchestrator → Debugger → Validator
Test Generation	Source files	Unit/integration test suite	Orchestrator → Test Agent → Executor
Code Review	Pull request / diff	Annotated feedback + fixes	Orchestrator → Reviewer → Planner
Full App Deployment	QA-approved codebase	Live URL + CI/CD pipeline	Orchestrator → DevOps Agent

2. Complete Agent Catalog — All 11 Agents

This platform uses exactly 11 specialized AI agents. Each agent has a single responsibility, a defined LLM model, and communicates exclusively through the Orchestrator's message bus. Below is the complete catalog.

> Key Design Principle: Every agent is stateless. State is stored externally in Redis (ephemeral) and PostgreSQL (persistent). Agents communicate by passing structured AgentMessage objects — they do not call each other directly.

1. Orchestrator Agent

Trigger: User submits a task or PRD
Model: GPT-4o (planning mode)
Input: User requirement, session context
Output: Sequenced agent invocations, final result

2. PM (Product Manager) Agent

Trigger: New project task received
Model: GPT-4o
Input: Raw PRD / markdown / docx / PDF
Output: Structured JSON spec (features, routes, schemas)

3. Architect Agent

Trigger: PM Agent delivers JSON spec
Model: GPT-4o
Input: JSON spec from PM Agent
Output: Tech stack, folder structure, atomic task list

4. Frontend Developer Agent

Trigger: Frontend task assigned by Architect
Model: GPT-4o / Claude 3.5
Input: Task + RAG context + constraints
Output: React component code / unified diff

5. Backend Developer Agent

Trigger: Backend task assigned by Architect
Model: GPT-4o / Claude 3.5
Input: Task + RAG context + API schema
Output: FastAPI route / DB model / service code

6. QA Agent

Trigger: Developer Agent completes a feature
Model: GPT-4o
Input: Generated code + test suite
Output: Bug report or pass signal + test results

7. Debugger Agent

Trigger: Validation or QA failure
Model: GPT-4o
Input: Error logs + failing code + attempt history
Output: Fixed code + root cause + prevention note

8. Validator Agent

Trigger: Code generated or fixed
Model: GPT-4o mini
Input: Code + test suite + build output
Output: Pass/fail report with scores per check type

9. Reviewer Agent

Trigger: PR created after task completion
Model: Claude 3.5 Sonnet
Input: Full diff + codebase context + guidelines
Output: Inline review comments + suggested fixes

10. Test Agent

Trigger: No tests found for modified module
Model: GPT-4o
Input: Source files + function signatures
Output: Unit + integration + edge case test files

11. DevOps Agent

Trigger: QA-approved codebase ready
Model: GPT-4o
Input: Final codebase + infra requirements

Output: Dockerfile, CI/CD YAML, deployed app URL

3. How Agents Connect — Communication Architecture

This is the most critical section for understanding how the platform works. Agents do NOT call each other directly. All communication flows through a central message bus managed by the Orchestrator Agent. Here is the complete connection model.

3.1 Agent Framework Selection

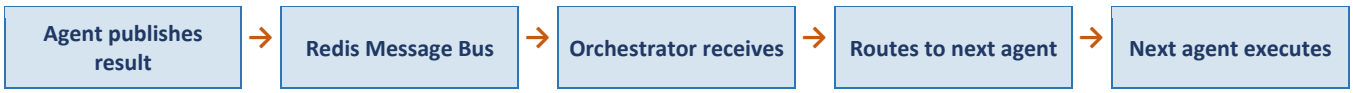
The platform uses CrewAI as the primary agent framework for role-based orchestration, with AutoGen available as an alternative for conversational multi-agent debugging workflows.

Framework	Best For	Our Use Case	Why Chosen
CrewAI	Role-based agents with defined goals and tools	PM → Architect → Developer → QA pipeline	Best fit for sequential + parallel crew workflows
AutoGen	Conversational multi-agent debugging loops	Debugger ↔ Validator back-and-forth	Excellent for iterative fix-and-retest patterns
LangChain Agent Executor	Low-level tool-calling and chain composition	RAG Engine, individual tool calls	Used for building individual agent tools

Decision: CrewAI manages the high-level workflow (PM → Architect → Developers → QA → DevOps). AutoGen manages the inner debug loop (Debugger ↔ Validator). LangChain provides the tool layer for both.

3.2 Central Message Bus Architecture

All agent-to-agent communication passes through a Redis-backed message bus. The Orchestrator publishes tasks; agents consume, execute, and publish results back. No agent has a direct reference to another agent.




```
# How agents communicate — via Redis message bus
# Publisher (any agent completing a task):
import redis
r = redis.Redis.from_url(REDIS_URL)
r.xadd('agent:results', {
    'job_id': job_id,
    'from_agent': 'pm_agent',
    'to_agent':   'orchestrator',
    'status':     'done',
    'payload':    json.dumps(structured_spec),
})
```

```
# Orchestrator consumer loop:
while True:
    messages = r.xread({'agent:results': '$'}, block=0, count=10)
    for stream, entries in messages:
        for msg_id, data in entries:
            orchestrator.handle_result(data)
```

3.3 Complete Agent Connection Map

The table below shows every valid connection between agents. ▶ = Orchestrator dispatches task to agent. → = Agent result triggers next agent. ↩ = Agent re-invokes a previous agent (debug loop). ◀ = Agent reports back to Orchestrator.

FROM \ TO	Orch	PM	Arch	FE Dev	BE Dev	QA	Debug	Valid	Review	Test	DevOps
Orchestrator		▶	▶	▶	▶	▶	▶	▶	▶	▶	▶
PM Agent	◀		→								
Architect	◀			→	→						
FE Developer	◀					→		→			
BE Developer	◀					→		→			
QA Agent	◀						→				
Debugger	◀			↩	↩						
Validator	◀						→				
Reviewer	◀										
Test Agent	◀							→			
DevOps	◀										

3.4 Agent Handoff Protocol — Step by Step

Every handoff follows this 5-step protocol to ensure traceability and fault tolerance:

Step 1 ▼ **Task Dispatch**
Orchestrator creates an AgentMessage, stores it in PostgreSQL (job record), and publishes to Redis stream 'agent:tasks:{agent_type}'.

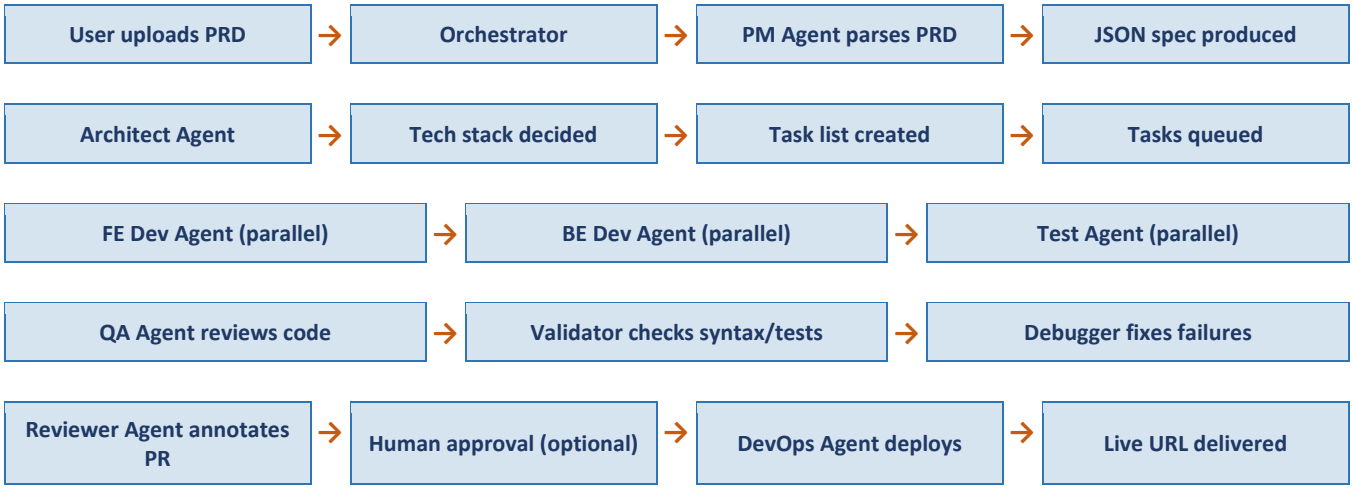
Step 2 ▼ **Agent Pickup**
Celery worker assigned to that agent type picks up the task from the Redis stream via a Celery consumer.

Step 3 ▼ **Execution**
Agent runs its LLM call + tool calls. All intermediate steps are logged to PostgreSQL audit log and streamed to UI via WebSocket.

Step 4	**Result Publication** Agent publishes result to Redis stream 'agent:results'. Result payload contains output + status + token count + duration.
Step 5	**Orchestrator Routing** Orchestrator reads the result, updates the job state in PostgreSQL, decides the next agent to invoke based on the current workflow stage and result status.

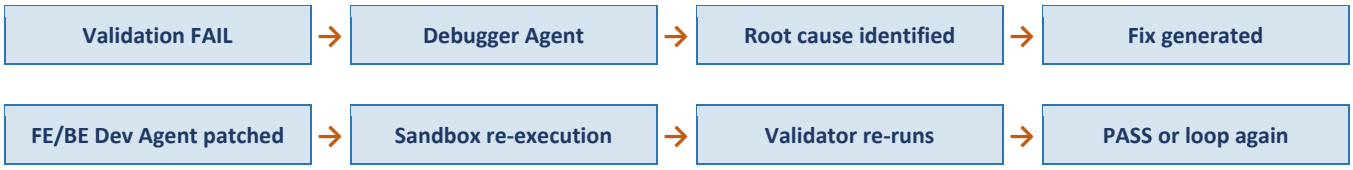
3.5 Full Workflow — New Project from PRD

Here is the complete end-to-end agent sequence for creating a new project from a PRD document:



3.6 Debug Loop — Agent Re-Invocation

When validation fails, the Debugger Agent is invoked and can re-trigger the relevant Developer Agent. This loop has a maximum of 5 iterations:



> Convergence guard: If the identical error is detected on 2 consecutive retries, or if the diff size shows no meaningful reduction, the loop terminates and a human escalation flag is set.

4. Prerequisites

4.1 Required Technical Skills

Skill	Level	Used For
Python 3.11+	Intermediate–Advanced	All backend services, agent logic, API server, async processing
FastAPI	Intermediate	REST API server, WebSocket, dependency injection, async routes
CrewAI / AutoGen	Intermediate	Agent framework — defining crews, roles, tools, and task flows
React (TypeScript)	Intermediate	Frontend IDE, real-time log display, Monaco Editor integration
Docker	Intermediate	Sandbox creation, resource isolation, container lifecycle
Git / GitHub API	Intermediate	Branch management, commits, PR creation, webhooks
PostgreSQL + pgvector	Basic–Intermediate	Job state, user data, repo metadata, vector embeddings
Redis + Celery	Basic	Message bus, job queues, pub/sub for real-time updates
LLM Concepts	Basic	Tokens, context windows, prompt structure, temperature, tool calling
SQLAlchemy / Alembic	Basic	ORM for Postgres, database schema migrations

4.2 Infrastructure Requirements

Component	Minimum	Recommended	Notes
CPU	4 vCPU	8 vCPU	Agent workers are CPU-intensive; more cores = more parallel jobs
RAM	8 GB	16–32 GB	Each sandbox ~512 MB; Python agents ~256 MB each
Storage	50 GB SSD	200 GB SSD	Docker images + repo indexes + pgvector data
OS	Ubuntu 22.04	Ubuntu 22.04 LTS	Best Docker + Python support
Network	100 Mbps	1 Gbps	LLM API calls are latency-sensitive
GPU (optional)	None	NVIDIA T4+	For local LLM inference via Ollama

4.3 Development Environment Setup

```
# 1. Install Python 3.11+ via pyenv
pyenv install 3.11.9 && pyenv global 3.11.9

# 2. Install Docker Desktop (Mac/Windows) or Docker Engine (Linux)

# 3. Start Redis via Docker
docker run -d -p 6379:6379 redis:alpine

# 4. Start PostgreSQL with pgvector
docker run -d -p 5432:5432 -e POSTGRES_PASSWORD=pass -e POSTGRES_DB=agentdb
ankane/pgvector:latest

# 5. Create virtual environment and install all dependencies
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

```
# 6. Install agent frameworks
pip install crewai autogen-agentchat langchain openai anthropic

# 7. Run database migrations
alembic upgrade head

# 8. Start API server
uvicorn app.main:app --reload --port 8000

# 9. Start Celery workers (one per agent type, or pooled)
celery -A app.worker worker --loglevel=info --
queues=orchestrator,pm,architect,developer,qa,debugger
```

5. Technical Dependencies

5.1 Python Backend Dependencies

Category	Package	Version	Purpose
Runtime	Python	3.11+	Core runtime for all backend services and agents
Framework	FastAPI	0.110+	Async REST API server with OpenAPI docs
ASGI Server	Uvicorn	0.29+	Production-grade ASGI server for FastAPI
Agent Framework	crewai	0.28+	Role-based multi-agent crew management
Agent Framework	pyautogen	0.2+	Conversational multi-agent debug loops
LLM Tooling	langchain + langchain-openai	0.1+	Agent tool building, chain composition
Real-time	python-socketio	5.x	WebSocket server for live agent log streaming
Job Queue	celery + redis	5.x	Distributed task queue for agent jobs
ORM	sqlalchemy + alembic	2.x / 1.x	Type-safe DB access and migrations
LLM Client	openai + anthropic	Latest	GPT-4o and Claude API calls
Embeddings	openai (text-embedding-3-small)	Latest	Vector embeddings for RAG pipeline
Vector DB	pgvector	0.5+	Similarity search inside PostgreSQL
Container SDK	docker	7.x	Docker API for sandbox management
Git Client	gitpython	3.x	Programmatic Git operations
AST Parser	tree-sitter	0.21+	Code parsing for 20+ languages
HTTP Client	httpx	0.27+	Async HTTP for webhooks and APIs
Validation	pydantic	2.x	Schema validation and serialization

Category	Package	Version	Purpose
Auth	python-jose + passlib	Latest	JWT auth + password hashing
Logging	structlog	24.x	Structured JSON logging for all services
Testing	pytest + pytest-asyncio	Latest	Unit and integration testing
Security	semgrep + bandit	Latest	Static security analysis of generated code
Doc Parsing	python-docx + pypdf + markdown	Latest	Parse PRD documents for PM Agent
Circuit Breaker	tenacity	8.x	Retry logic and circuit breaker for LLM calls

5.2 Frontend Dependencies

Package	Version	Purpose
React + TypeScript	18 / 5.x	Frontend IDE and user interface
Vite	5.x	Fast frontend build tool and dev server
Monaco Editor	0.44+	VS Code-grade in-browser code editor
Socket.io-client	4.x	WebSocket client for real-time agent log streaming
TanStack Query	5.x	Server state management and API data fetching
Zustand	4.x	Lightweight client state management
Tailwind CSS	3.x	Utility-first CSS framework
shadcn/ui	Latest	Accessible UI component library

6. Required Services

6.1 Free / Open Source Services

Service	Role	Setup Method
PostgreSQL 15+ with pgvector	Primary database + vector similarity search	Docker: ankane/pgvector:latest
Redis 7+	Message bus (agent results stream), Celery broker, session cache	Docker: redis:7-alpine
Docker Engine	Execution sandbox runtime for all supported languages	System install (docker.io)
Ollama (optional)	Local LLM inference — Llama 3, Mistral, CodeLlama	ollama.ai local install
ChromaDB (optional)	Lightweight local vector DB for development	pip install chromadb
Gitea (optional)	Self-hosted Git server for isolated repo management	Docker: gitea/gitea

6.2 Paid / Cloud Services (Recommended for Production)

Service	Purpose	Approx. Cost	Open Source Alternative
OpenAI GPT-4o	Primary LLM — all developer and planning agents	$5–15 / 1M tokens	Ollama + Llama 3.1 70B
Anthropic Claude 3.5 Sonnet	Code review (Reviewer Agent), secondary LLM	$3–15 / 1M tokens	GPT-4o
E2B Sandbox	Managed secure code execution as alternative to Docker	$0.10 / hour	Self-hosted Docker
Pinecone	Managed vector DB for very large repos (>5M LOC)	$70+/month	pgvector (free)
GitHub / GitLab API	PR creation, webhook triggers, code hosting	Free tier sufficient	Gitea (self-hosted)
AWS / GCP / Azure VM	Production hosting for all platform services	$50–200/month	Hetzner (cheaper, EU)
Datadog / Grafana Cloud	Observability, metrics, alerting, APM	$0–$70/month	Self-hosted Prometheus + Grafana

7. Team & Human Resources

7.1 Core Team Roles

Role	Count	Stage	Responsibilities	Key Skills
Backend / AI Engineer	2–3	All stages	Python agent logic, FastAPI, LLM integration, CrewAI/AutoGen setup, prompt engineering	Python, FastAPI, CrewAI, OpenAI SDK, Docker
Frontend Engineer	1	Phase 3+	React IDE, Monaco Editor, real-time log UI, WebSocket client, PRD upload UI	React, TypeScript, Monaco, Socket.io
DevOps / Platform Engineer	1	Phase 4+	Docker, Kubernetes, CI/CD pipelines, Redis cluster, observability	Docker, K8s, Terraform, Prometheus
QA / Test Engineer	1	Phase 3+	Integration tests, agent loop regression, sandbox testing, contract tests	pytest, Selenium, API testing
Product / Tech Lead	1	All stages	Architecture decisions, roadmap, code review, agent design, stakeholder alignment	System design, Python, LLM architecture
Security Engineer (part-time)	0.5	Phase 4+	Sandbox security review, threat modeling, secret scanning	Docker security, OWASP, semgrep

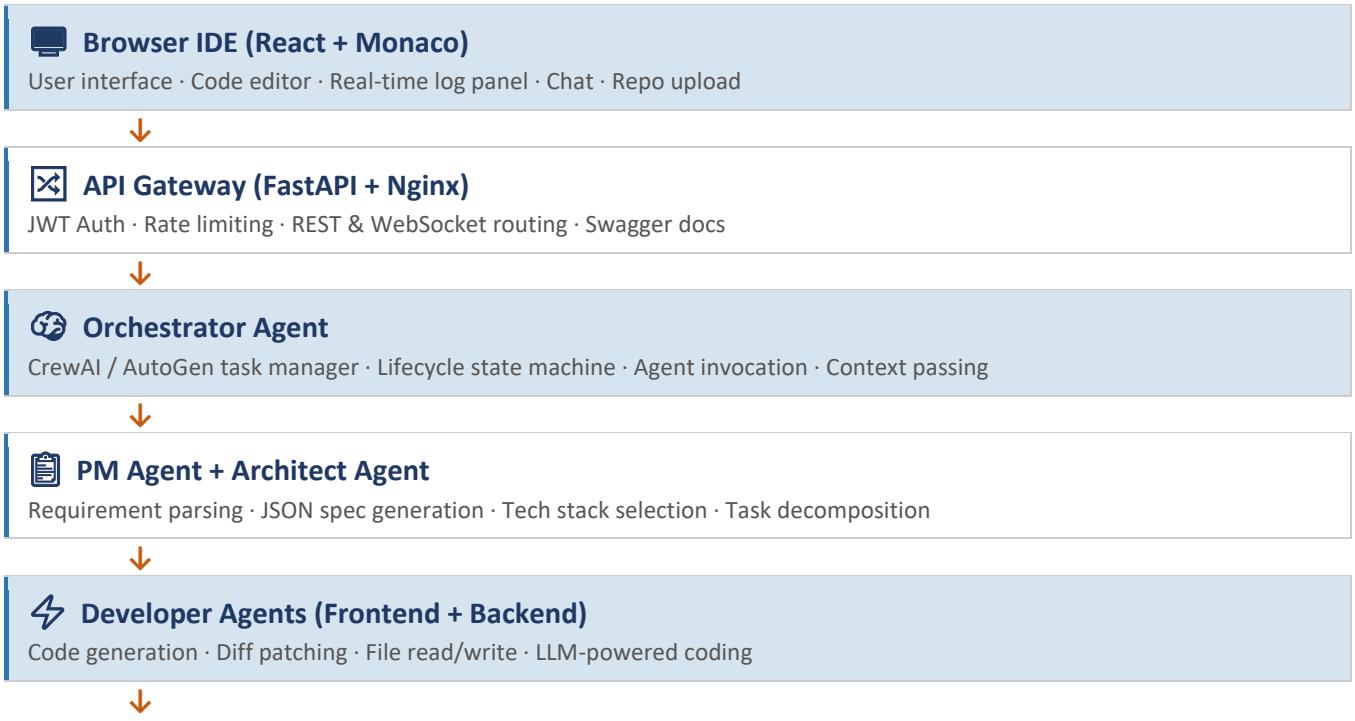
7.2 Development Tools & Licenses

Tool	Type	Purpose	Cost
GitHub / GitLab	SaaS	Source control, CI/CD, PR management	Free–$19/user/month
GitHub Actions / GitLab CI	SaaS	Automated testing and deployment pipelines	Free tier / usage-based
PyCharm / VS Code	IDE	Python development environment	Free (VS Code) / $249/yr (PyCharm)
Postman / Insomnia	Testing	API development and testing	Free
Jira / Linear	PM	Issue tracking, sprint planning, backlog management	$8–10/user/month
Confluence / Notion	Docs	Technical documentation, architecture decisions	$5–8/user/month
Sentry	Monitoring	Error tracking for production services	Free–$26/month
1Password / HashiCorp Vault	Security	Secret management for API keys and credentials	$3–$0.03/secret/month

8. System Architecture

8.1 High-Level Architecture

All system layers from the user's browser IDE down to the data infrastructure, showing where each agent lives:



QA Agent + Validator Agent
Unit tests · Linting · Security scan · Semantic LLM check · Bug reports

↓

Debugger Agent
Error analysis · Root cause identification · Auto-fix loop (max 5 retries)

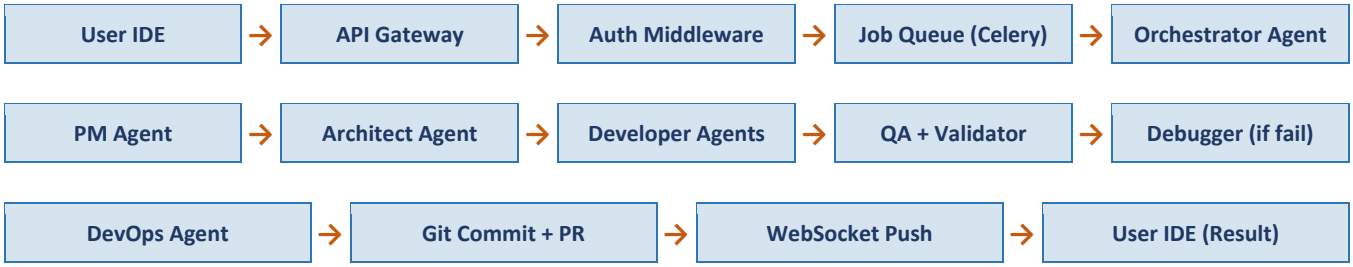
↓

DevOps Agent
Dockerfile generation · CI/CD config · Cloud deployment via CLI · URL delivery

↓

Data & Infrastructure Layer
PostgreSQL + pgvector · Redis (Celery) · Docker Sandbox · GitHub API · Prometheus

8.2 Request Flow



8.3 Service Breakdown

Service	Technology	Responsibility	Scales?
API Gateway	FastAPI + Nginx	Auth, rate limiting, routing to workers	Horizontal
Agent Orchestrator	CrewAI + Celery workers	Task planning, crew coordination, state machine	Horizontal
LLM Engine	OpenAI/Anthropic Python SDK	LLM calls, prompt assembly, response parsing	By API quota
RAG Engine	pgvector + tree-sitter	Repo indexing, semantic search, context retrieval	Horizontal
Patch Generator	Python difflib + subprocess	Generate minimal diffs, apply to filesystem	Horizontal
Execution Sandbox	Docker + docker-py	Isolated code execution with timeouts	Horizontal
Validation Layer	pylint/mypy/pytest/semgrep	Multi-type correctness checks	Horizontal
Message Bus	Redis Streams (xadd/xread)	All inter-agent communication	Redis cluster
State Manager	Redis + PostgreSQL	Ephemeral + persistent state	Redis clusters
Frontend IDE	React + Monaco	Code editor, live logs, chat interface	CDN/static

Service	Technology	Responsibility	Scales?
Notification Service	python-socketio	Real-time agent logs streamed to UI	Horizontal

8.4 Pre-Execution Guardrails

- File existence validation — confirm target files exist in the workspace
- Language/runtime detection — auto-detect Python, JS, Go, etc. via tree-sitter
- Syntax sanity check — fast tree-sitter parse before LLM is invoked
- Dependency presence — check for requirements.txt, package.json, pyproject.toml
- Token budget check — verify job has remaining token budget before proceeding

> Guardrails run in under 100ms and prevent up to 30% of unnecessary LLM calls, significantly reducing per-job cost.

9. Agent Architecture — Deep Dive

9.1 Orchestrator Agent

The Orchestrator is the central project manager. It never generates code itself — it only plans, delegates, and monitors.

Attribute	Detail
Framework	CrewAI Crew Manager + custom Python state machine
Model	GPT-4o (with function calling enabled)
Trigger	User submits a task via the API or IDE
Responsibilities	Initialize job, invoke agents in sequence, pass context between agents, monitor retries, update job state, stream progress to UI
Tools	Redis publisher (publish tasks), PostgreSQL writer (log state), WebSocket emitter (stream logs)
State stored in	PostgreSQL: job record with full history | Redis: current active subtask

9.2 PM (Product Manager) Agent

Translates human-readable documents into machine-readable structured specs that all downstream agents can consume.

Attribute	Detail
Model	GPT-4o
Trigger	New project task received by Orchestrator
Input	Raw document (Markdown, DOCX, PDF, plain text) uploaded by user

Attribute	Detail
Output	Structured JSON spec: { features, user_stories, data_models, api_endpoints, ui_components }
Tools	python-docx (read .docx), pypdf (read .pdf), markdown parser, clarifying_question_tool (ask user)
Clarification	If document is ambiguous, PM Agent generates targeted questions for the user before proceeding
Connects to	Orchestrator (reports back), Architect Agent (passes spec)

```
# PM Agent output schema (Pydantic)
class PMSpec(BaseModel):
    project_name: str
    features: List[Feature]
    user_stories: List[UserStory]
    data_models: List[DataModel]     # e.g. User: {id, email, created_at}
    api_endpoints: List[APIEndpoint] # e.g. POST /api/login -> JWT
    ui_components: List[UIComponent] # e.g. LoginForm with email/password
    tech_preferences: Optional[dict] # from document or user input
    ambiguities: List[str]           # questions to clarify before proceeding
```

9.3 Architect Agent

Makes all high-level technical decisions and decomposes the PM spec into an ordered, dependency-aware task list.

Attribute	Detail
Model	GPT-4o
Trigger	PM Agent delivers structured JSON spec
Input	PMSpec JSON from PM Agent
Output	ArchPlan: { tech_stack, folder_structure, task_list (ordered, with dependencies) }
Decisions made	Web framework (React/Next.js/Vue), Backend (FastAPI), DB (PostgreSQL/MongoDB), Auth strategy
Task breakdown	Decomposes each feature into atomic tasks: 'Create React LoginForm component', 'Create POST /api/login FastAPI route'
Connects to	Orchestrator (reports back), FE Developer Agent (sends frontend tasks), BE Developer Agent (sends backend tasks)

9.4 Developer Agents (Frontend + Backend)

Two specialized developer agents run in parallel — one for frontend, one for backend — each with language-specific tools and context.

Attribute	Frontend Dev Agent	Backend Dev Agent
Model	GPT-4o / Claude 3.5	GPT-4o / Claude 3.5
Task types	React components, routing, state management, CSS	FastAPI routes, DB models, business logic, APIs

Attribute	Frontend Dev Agent	Backend Dev Agent
Tools	write_file, read_file, run_npm_command, read_codebase_context	write_file, read_file, run_pip_command, read_codebase_context, run_db_migration
Output format	Full file content OR unified diff patch	Full file content OR unified diff patch
Context window	Current component tree + RAG chunks (8K tokens max)	Existing routes + DB schema + RAG chunks (8K tokens max)
Connects to	Orchestrator, QA Agent, Validator Agent	Orchestrator, QA Agent, Validator Agent

9.5 QA Agent

Acts as a software tester — attempts to find bugs by running the application in a test environment and analyzing behavior.

Attribute	Detail
Model	GPT-4o
Trigger	Developer Agent marks a feature as complete
Input	Generated code + existing test suite + feature spec from PM Agent
Actions	Run the sandbox, execute integration tests, perform static code analysis (pylint/eslint), check for edge cases
Output	BugReport: { severity, file, line, description, reproduction_steps } OR PassSignal
On bugs found	Publishes bug report to Orchestrator, which routes to Debugger Agent
Connects to	Orchestrator, Debugger Agent (sends bug reports)

9.6 Debugger Agent

Attribute	Detail
Model	GPT-4o
Trigger	Validation or QA failure; receives error output + failing code
Input	Error logs, failing code, test output, attempt history (to prevent identical retries)
Output	{ root_cause, fix (unified diff), explanation, prevention_note }
Loop limit	Maximum 5 iterations; convergence detected if same error repeats 2x
Framework	AutoGen multi-agent conversation — Debugger ↔ Validator have a back-and-forth to verify fix
Connects to	Orchestrator, FE/BE Developer Agents (sends fix patch), Validator Agent (re-runs checks)

9.7 DevOps Agent

Handles all deployment concerns — containerization, CI/CD pipeline generation, and cloud deployment. This agent only activates after QA has fully passed.

Attribute	Detail
Model	GPT-4o
Trigger	QA Agent issues a full PassSignal for the complete codebase
Input	Final codebase + infrastructure requirements (from Architect Agent's ArchPlan)
Output	Dockerfile, .github/workflows/main.yml (CI/CD), deployed application URL
Tools	write_file (Dockerfile/YAML), run_terminal (docker build, vercel deploy, netlify deploy, aws CLI)
Supported targets	Vercel (frontend), Railway / Render (backend), AWS ECS / GCP Cloud Run, self-hosted VPS
Connects to	Orchestrator (reports deployed URL), User IDE (delivers final result)

9.8 End-to-End Agent Loop (Existing Codebase)

Step 1	**Planner Agent (GPT-4o)** Breaks task into precise, atomic subtasks with a JSON execution plan and dependency graph.
Step 2	**RAG Engine** Retrieves relevant code context from the indexed repository using hybrid semantic + keyword search.
Step 3	**Executor Agent (GPT-4o / Claude 3.5)** Generates code or unified diff patch based on subtask + RAG context + constraints.
Step 4	**Patch Generator** Validates diff format, runs git apply --check (dry run), applies patch to sandbox filesystem.
Step 5	**Execution Sandbox (Docker)** Runs the modified code in an isolated container with resource limits and network isolation.
Step 6	**Validator Agent (GPT-4o mini)** Checks syntax, runs tests, performs security scan, and LLM semantic validation.
Step 7	**Debugger Agent (GPT-4o) — on failure** Analyzes error output, patches the fix, retries execution loop (max 5 iterations).
Step 8	**Git Commit + PR** Pushes feat/agent-{jobId} branch, creates PR with structured commit messages and auto-generated description.
Step 9	**WebSocket Push to UI** Delivers real-time result, logs, patch diff, and validation report to the user's IDE.

9.9 Agent Roles Summary Table

Agent	LLM Model	Trigger	Input	Output	Framework
Orchestrator	GPT-4o	User task submitted	Task + session context	Agent invocations	CrewAI Manager
PM Agent	GPT-4o	New project task	PRD document (any format)	Structured JSON spec	CrewAI Agent

Agent	LLM Model	Trigger	Input	Output	Framework
Architect Agent	GPT-4o	PM spec ready	JSON spec	Tech stack + task list	CrewAI Agent
Frontend Dev Agent	GPT-4o / Claude 3.5	Frontend task assigned	Task + RAG context	React code / diff	CrewAI + LangChain tools
Backend Dev Agent	GPT-4o / Claude 3.5	Backend task assigned	Task + API schema + RAG	FastAPI code / diff	CrewAI + LangChain tools
QA Agent	GPT-4o	Feature completed	Code + spec + tests	Bug report or pass	CrewAI Agent
Debugger Agent	GPT-4o	Validation failure	Error logs + code + history	Fix + root cause	AutoGen conversation
Validator Agent	GPT-4o mini	Code generated/fixed	Code + build + tests	Pass/fail + score	LangChain Agent
Reviewer Agent	Claude 3.5 Sonnet	PR created	Full diff + context	Inline comments	CrewAI Agent
Test Agent	GPT-4o	No tests found	Source files + signatures	Test files	CrewAI Agent
DevOps Agent	GPT-4o	QA full pass	Codebase + infra reqs	Dockerfile + deploy URL	CrewAI + CLI tools

10. Prompt Engineering

10.1 Prompt Design Principles

- Role clarity — every prompt starts with explicit agent persona and task scope
- Context injection — RAG chunks injected between <context> tags to avoid confusion with instructions
- Output format enforcement — always specify exact JSON schema or code block format
- Constraint declaration — list what the agent must NOT do (no hallucinated imports, no full rewrites)
- Chain-of-thought — for planning prompts, request step-by-step reasoning before final output
- Temperature tuning — Planner: 0.3, Executor: 0.1, Debugger: 0.2, PM Agent: 0.2

10.2 PM Agent Prompt Template

```
System: You are an expert Product Manager AI.
Parse the provided document and extract all structured information.
If anything is ambiguous, add it to the 'ambiguities' list — do NOT assume.

User:
Document content: {document_text}
User tech preferences (if any): {tech_preferences}

Respond ONLY with valid JSON matching this schema:
{
  'project_name': '...',
```

```
  'features': [{'id': 1, 'name': '...', 'description': '...', 'priority':
'high|med|low'}],
  'user_stories': [{'as_a': '...', 'i_want': '...', 'so_that': '...'}],
  'data_models': [{'name': '...', 'fields': [{'name': '...', 'type': '...'}]}],
  'api_endpoints': [{'method': 'POST', 'path': '/api/...', 'request': {}, 'response':
{}}],
  'ui_components': [{'name': '...', 'page': '...', 'elements': [...]}],
  'ambiguities': ['Question 1 for user', 'Question 2 for user']
}
```

10.3 Architect Agent Prompt Template

```
System: You are a senior software architect.
Given the product specification, make all technology decisions and create a detailed task
list.

User:
Spec: {pm_spec_json}
Tech preferences: {tech_preferences}

Respond ONLY with valid JSON:
{
  'tech_stack': { 'frontend': 'React + Vite', 'backend': 'FastAPI', 'db': 'PostgreSQL',
'auth': 'JWT' },
  'folder_structure': { 'frontend/src/': [...], 'backend/app/': [...] },
  'tasks': [{
    'id': 1, 'agent': 'frontend|backend',
    'type': 'create|modify|test',
    'file': 'path/to/file',
    'description': '...',
    'dependencies': []
  }]
}
```

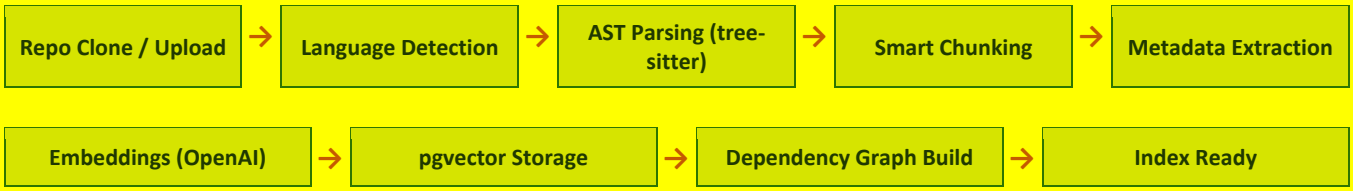
10.4 Execution Prompt, Debug Prompt, Validation Prompt

```
# EXECUTION PROMPT
System: You are an expert {language} developer. Write ONLY the requested code.
Follow existing code style exactly. No explanations unless in comments.
User:
Task: {subtask_description} | File: {file_path}
Existing code: {current_code} | Context: {rag_chunks}
Output: unified diff (diff -u format) or full file in triple backticks.

# DEBUG PROMPT
System: You are a debugging expert. Identify root cause and provide minimal fix.
User:
Code: {code} | Error: {error_logs} | Test: {test_code} | Attempts: {attempt_history}
Respond: { 'root_cause': '...', 'fix': 'diff or full file', 'prevention_note': '...' }

# VALIDATION PROMPT
System: You are a code quality reviewer.
User:
Code: {generated_code} | Requirement: {task} | Language: {language}
```

```
Respond: { 'syntax_valid': true, 'logic_correct': true, 'security_issues': [],
           'overall_score': 0-10, 'approved': true|false }
```

11. Repository Indexing Architecture

11.1 Indexing Pipeline



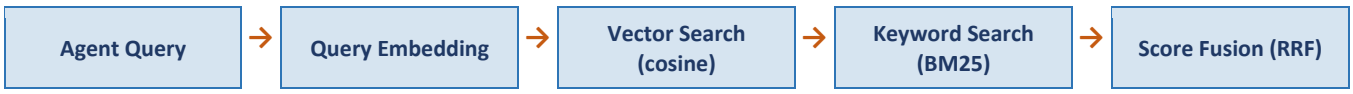
11.2 Chunking Strategy

Chunk Type	Trigger	Approx. Size	Metadata Stored
Function/method	AST function node	10–80 lines	name, args, return type, file, start/end line
Class definition	AST class node	Full class	class name, methods, properties, file path
Module/file header	First 30 lines	~30 lines	imports, exports, file purpose, dependencies
Comment block	JSDoc / docstring	Variable	associated function, type, description
Config file	json/yaml/toml/env files	Full file	key-value pairs, environment target
Test file	*.test.*, *.spec.*, test_*.py	Full function	test name, target function, assertions

11.3 Code Graph & Dependency Analysis

- Nodes: functions, classes, files, modules
- Edges: imports, function calls, inheritance, interface implementations
- Enables impact analysis before patching — the Planner Agent knows what else might break
- Supports 'find all usages' queries and dependency-aware RAG retrieval
- Stored in PostgreSQL relational tables for fast traversal

12. RAG Pipeline



12.1 Search Types

Type	Algorithm	Best For	Weakness
Vector search	Cosine similarity (pgvector)	Semantic meaning, paraphrased queries	Misses exact keyword matches
Keyword search	BM25 (pg_trgm in PostgreSQL)	Exact function/variable names	No semantic understanding
Hybrid (default)	Reciprocal Rank Fusion (RRF)	Best of both — used in all queries	Slightly higher latency
Graph-based (advanced)	Dependency graph traversal	Transitive code dependencies	Complex to build, higher memory

12.2 Task-Aware Retrieval Strategy

Task Type	Retrieval Strategy	Vector Weight	Keyword Weight	Graph Weight
Bug fix	Error-local context prioritized	60%	30%	10%
Refactor	Dependency graph traversal included	40%	20%	40%
New feature	Interfaces, patterns, similar modules	70%	20%	10%
Test generation	Target function + related helpers	50%	40%	10%

13. Patch Generation System

13.1 Why Patches Over Full Rewrites

Approach	Safety	Review Cost	Merge Conflicts	LLM Tokens
Full file rewrite	Low — overwrites history	High — full review needed	High risk	High cost
Unified diff patch	High — only changes lines	Low — focused review	Low risk	Low cost
AST-level patch (ideal)	Highest — structure-aware	Medium	Very low	Medium

13.2 Patch Workflow

1. Executor Agent outputs unified diff format (diff -u)

2. Patch validated for format correctness before apply
3. Applied via git apply --check (dry run) first
4. On dry run success: git apply --whitespace=fix to sandbox
5. On apply failure: fallback to full file rewrite with diff highlighted
6. Applied patch written to job record in PostgreSQL for full audit trail

14. Execution Sandbox

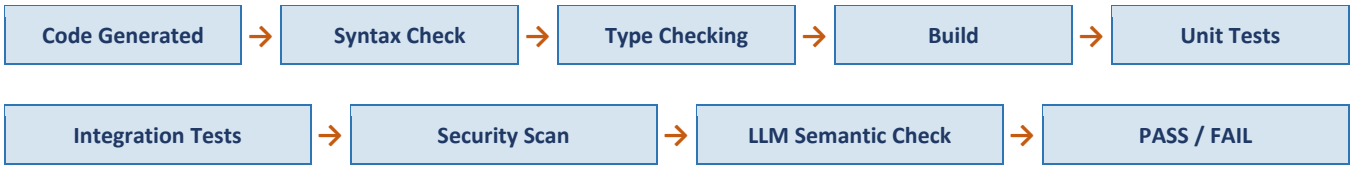
14.1 Security Architecture

Control	Implementation	What It Blocks
Network isolation	Docker --network=none by default	Exfiltration, SSRF attacks, outbound requests
CPU limit	--cpus=0.5 per container	CPU exhaustion attacks
Memory limit	--memory=512m	Memory exhaustion / OOM conditions
Filesystem	Read-only host mount + tmpfs workspace	Host filesystem access and data leakage
Process limit	--pids-limit=100	Fork bombs and process explosion
Time limit	30s hard timeout via SIGKILL	Infinite loops and hung processes
User isolation	Non-root user (uid=1000) inside container	Privilege escalation attempts
Capabilities	--cap-drop=ALL	Kernel exploit attempts
Seccomp	Custom seccomp profile	Dangerous syscalls (ptrace, mount, etc.)

14.2 Supported Runtimes

Language	Base Image	Build Command	Test Command
Python	python:3.12-slim	pip install -r requirements.txt	pytest
Node.js	node:20-alpine	npm install	npm test
Java	eclipse-temurin:21-alpine	mvn package	mvn test
Go	golang:1.22-alpine	go build ./...	go test ./...
Rust	rust:1.77-slim	cargo build	cargo test
PHP	php:8.3-cli-alpine	composer install	phpunit
Ruby	ruby:3.3-alpine	bundle install	rspec

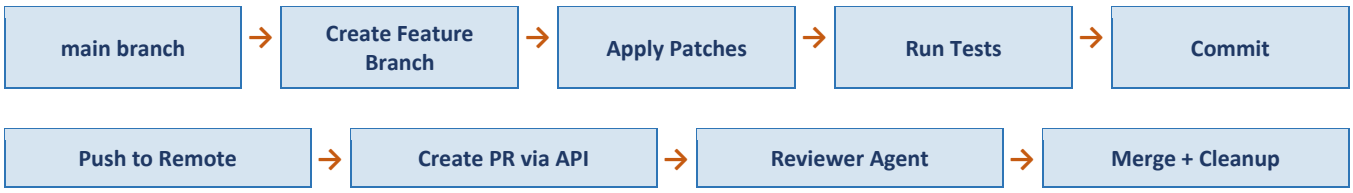
15. Validation Layer



15.1 Validation Types

Type	Tool (Python stack)	When	Blocks Deploy?
Syntax	tree-sitter / py_compile / pylint --syntax-only	Immediately after code generation	Yes
Type checking	mypy (Python), tsc (TypeScript)	After syntax pass	Yes
Build	subprocess: pip install / go build / mvn compile	After type check	Yes
Unit tests	pytest / Jest / JUnit	After build	Yes
Integration tests	pytest + FastAPI TestClient	After unit tests	Yes
Security scan	semgrep + bandit + npm audit	Parallel with tests	High severity only
Linting / style	pylint, flake8, black, ESLint	Parallel with tests	No (warning only)
Semantic check	LLM-based requirement match	After all tests pass	Low confidence only
Behavioral regression	Custom snapshot / output diff	After tests pass	Yes (if deviation)

16. Git Integration



16.1 Commit Convention

```
# Format: {type}({scope}): {description} [agent-generated]
feat(auth):     add JWT refresh token rotation [agent-generated]
fix(api):       handle null response from payment gateway [agent-generated]
test(user):     add unit tests for UserService.create_user [agent-generated]
refactor(db):   extract query builder to separate module [agent-generated]
```

17. State Management

State Type	Storage	TTL	Example Data
Job queue	Redis (Celery broker)	Until processed	Pending/running job IDs and payloads
Agent context	Redis hash	Duration of job	Current subtask, retry count, last output
Agent messages	Redis Streams (xadd)	24 hours	All inter-agent messages for current job
Conversation history	Redis list	24 hours	LLM message history for multi-turn context
File system state	Disk (sandbox volume)	Duration of job	Working copy of modified files
Job record	PostgreSQL (SQLAlchemy)	Permanent	Full job metadata, status, result, cost
Repo index	PostgreSQL (pgvector)	Permanent (versioned)	Embeddings per chunk with commit SHA
Audit log	PostgreSQL	Permanent	Every agent action, decision, prompt, output
User sessions	Redis	24 hours	JWT token, preferences, rate limit counters

18. Cost Control & Optimization

Control	Mechanism	Default Limit
Max LLM steps per job	Step counter in orchestrator	20 steps
Token budget per job	Token counter with hard stop	200,000 tokens
Monthly budget per user	Redis counter + PostgreSQL ledger	Configurable by plan
Model routing	GPT-4o mini for validation; GPT-4o for coding	~60% savings on validation
RAG context limit	Max 8K tokens per LLM call	Reduces per-call cost
Prompt caching	Redis cache for identical prompts (SHA-256)	~20% cache hit rate typical
Batch embeddings	Embed chunks in batches of 100 via OpenAI batch API	~50% reduction in embedding calls

18.1 Cost Estimation by Task Type

Task Type	Avg LLM Calls	Avg Tokens	Est. Cost (GPT-4o)	Est. Cost (Claude 3.5)
Simple bug fix	3–5	15,000	~$0.15	~$0.08
New feature (small)	8–12	50,000	~$0.50	~$0.25
New feature (medium)	15–25	120,000	~$1.20	~$0.60
New project (small)	30–50	300,000	~$3.00	~$1.50
New app from PRD	50–100	600,000	~$6.00	~$3.00

Task Type	Avg LLM Calls	Avg Tokens	Est. Cost (GPT-4o)	Est. Cost (Claude 3.5)
Large refactor	20–40	200,000	~$2.00	~$1.00

19. Observability

19.1 Logging

- Structured JSON logging via structlog — every agent action logged with job_id, agent_type, duration, tokens_used
- Log levels: ERROR, WARN, INFO, DEBUG — configurable per service via environment variable
- Logs shipped to ELK Stack or Datadog
- Sensitive data (API keys, user code) redacted via custom structlog processors

19.2 Metrics (Prometheus + Grafana)

Metric	Type	Alert Threshold
job_completion_rate	Gauge	< 80% in 10-minute window
agent_llm_latency_p95	Histogram	> 15 seconds
llm_token_usage_total	Counter	> 90% of monthly budget
sandbox_execution_timeout_total	Counter	> 5 per minute
validation_failure_rate	Gauge	> 40% over 5 minutes
active_jobs	Gauge	> 100 concurrent jobs
celery_queue_depth	Gauge	> 50 pending tasks

19.3 Distributed Tracing

- OpenTelemetry Python SDK on all services (opentelemetry-sdk + auto-instrumentation)
- Trace spans: HTTP → Orchestrator → each agent → LLM call → sandbox → validation
- Traces exported to Jaeger (self-hosted) or Datadog APM
- Sampling: 100% for errors, 10% for successful jobs

20. Failure Handling & Recovery

Failure Category	Detection	Recovery Strategy	Max Retries
LLM API rate limit (429)	HTTP status code	Exponential backoff: 1s, 2s, 4s, 8s	3
LLM API timeout	Request timeout > 30s	Switch to fallback model, retry	2

Failure Category	Detection	Recovery Strategy	Max Retries
LLM hallucination	JSON parse fail	Retry with stricter prompt + examples	3
Sandbox crash	Exit code != 0 or timeout	New container, re-run, send logs to Debugger	2
Patch conflict	git apply returns non-zero	Full file rewrite fallback	1
DB connection lost	SQLAlchemy error	Reconnect pool, circuit breaker open	5
Redis down	Celery connection error	Jobs persisted to PostgreSQL as fallback	Infinite
Agent infinite loop	Step counter exceeded	Hard stop, return partial result + escalation	0

20.1 Circuit Breaker Pattern

- All external calls (LLM APIs, Docker, DB) wrapped in circuit breaker via tenacity library
- CLOSED: normal operation. OPEN: after 5 failures — requests fail fast. HALF-OPEN: single probe after 30s

21. Infrastructure Architecture

21.1 Docker Compose (Development)

```
services:
  api-server:     # port 8000 — FastAPI + Uvicorn
    image: python:3.11-slim
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

  celery-worker:  # 3 replicas — one per agent group
    command: celery -A app.worker worker --queues=orchestrator,pm,architect,developer,qa
    deploy: { replicas: 3 }

  sandbox-service: # Docker-in-Docker
    volumes: ['/var/run/docker.sock:/var/run/docker.sock']

  frontend:       # port 5173 — Vite React
  postgres:       # port 5432 — PostgreSQL 15 + pgvector (ankane/pgvector:latest)
  redis:          # port 6379 — Redis 7 (broker + message bus + cache)
```

21.2 Production Kubernetes

Deployment	Replicas	Resources	HPA Trigger
api-server (FastAPI)	3 min	0.5 CPU / 512 MB RAM	CPU > 70%
celery-worker	5 min, 20 max	2 CPU / 2 GB RAM	Celery queue depth > 10

Deployment	Replicas	Resources	HPA Trigger
sandbox-service	3 min	1 CPU / 1 GB RAM	Active sandboxes > 50
frontend (nginx)	2 min	0.1 CPU / 128 MB RAM	CPU > 80%
postgres	1 primary + 2 replicas	4 CPU / 8 GB RAM	Manual only
redis	3-node cluster	2 CPU / 4 GB RAM	Manual only

21.3 Environment Variables

Variable	Required	Example Value
DATABASE_URL	Yes	postgresql+asyncpg://user:pass@localhost:5432/agentdb
REDIS_URL	Yes	redis://localhost:6379
CELERY_BROKER_URL	Yes	redis://localhost:6379/0
OPENAI_API_KEY	Yes	sk-...
ANTHROPIC_API_KEY	No	sk-ant-...
DOCKER_SOCKET	Yes	/var/run/docker.sock
JWT_SECRET	Yes	min 32 char random string
GITHUB_TOKEN	No	ghp_...
MAX_CONCURRENT_JOBS	No	10
LLM_TOKEN_BUDGET_PER_JOB	No	200000
SANDBOX_TIMEOUT_SECONDS	No	30
LOG_LEVEL	No	INFO
AGENT_FRAMEWORK	No	crewai (default) | autogen

22. Development Plan (Phase-by-Phase)

Phase	Timeline	Goals	Key Tasks
Phase 1	Weeks 1–2	Working API + Orchestrator + basic LLM integration	FastAPI, OpenAI/Anthropic SDK, Celery + Redis, PostgreSQL schema, basic Orchestrator state machine, job status API
Phase 2	Weeks 3–4	End-to-end loop for single-file tasks	Planner + Executor agents, Validator, Debugger, retry logic, Docker sandbox, WebSocket streaming, basic React UI
Phase 3	Weeks 5–7	PM Agent + Architect Agent + RAG pipeline	Document parsing (python-docx/pypdf), PM Agent with JSON spec output, Architect Agent with task decomposition, CrewAI crew setup, tree-sitter indexing, pgvector, hybrid search

Phase	Timeline	Goals	Key Tasks
Phase 4	Weeks 8–9	Frontend Dev + Backend Dev agents + patch workflow	CrewAI developer agents with tools, unified diff generation, git apply workflow, rollback, full validation pipeline, Test Agent, cost controls
Phase 5	Weeks 10–11	Full IDE + QA Agent + DevOps Agent	Monaco Editor, real-time log panel, QA Agent with sandbox testing, DevOps Agent with Dockerfile + CI/CD generation, deployment to Vercel/Railway
Phase 6	Week 12+	Production-ready, scalable platform	Kubernetes, Prometheus + Grafana, OpenTelemetry, circuit breakers, multi-model routing, user billing, multi-tenant isolation, AutoGen debug loop

23. Technical Challenges & Solutions

Area	Challenge	Root Cause	Solution	Status
AI	LLM hallucination	Insufficient context	RAG + constrained Pydantic output + retry with examples	Mitigated
AI	Inconsistent code style	LLM nondeterminism	Few-shot examples + temperature=0.1 + black/pylint enforcement	Mitigated
Agents	Agent framework complexity	CrewAI + AutoGen learning curve	Start with simple CrewAI crew (Phases 1–2), add AutoGen in Phase 6	Planned
Code	Breaking changes in repos	Missing context	Full RAG + code graph impact analysis before patch	Mitigated
Code	Language diversity	Too many runtimes	Pluggable Docker runtime images per language	In progress
Infra	Sandbox security escapes	Docker misconfiguration	Seccomp + cap-drop + non-root + network=none	Mitigated
Infra	Large repo indexing (>500K LOC)	Memory/time limits	Streaming AST + incremental indexing + background Celery tasks	In progress
Cost	Unpredictable LLM costs	Unbounded agent loops	Step counter + token budget + tiered model routing	Mitigated
UX	Slow feedback (>2 min)	LLM + sandbox latency	Streaming logs + intermediate results via WebSocket	In progress

24. Security Considerations

24.1 Threat Model

Threat	Vector	Mitigation
Prompt injection	Malicious user input in task description	Pydantic input sanitization; separate system/user context; output validation
Code execution escape	Malicious code in sandbox	Seccomp, non-root, --cap-drop=ALL, network=none
Secret exfiltration	Agent writes secrets to output	truffleHog + detect-secrets before commit; no network in sandbox
API key theft	Compromised frontend or logs	Keys only in env vars; never logged; server-side only; Vault for production
SSRF via agent	Agent makes internal network requests	network=none; URL allowlist for web-enabled jobs
Data leakage between users	Shared pgvector index	Separate pgvector schemas per organization (tenant isolation)
Context poisoning	Malicious instructions in code comments	Context sanitization + system prompt priority enforcement

24.2 Authentication & Authorization

- JWT-based auth: 15-minute access tokens + 7-day refresh tokens (python-jose)
- RBAC: Admin, Developer, Viewer — enforced via FastAPI dependencies
- Rate limiting: 100 req/min per user, 10 concurrent jobs per org (slowapi middleware)
- All repo access verified against org membership before indexing begins

25. Product Roadmap

Phase	Feature Set	Target Users	Milestone
MVP (Phase 1–2)	Basic agent loop, single-file tasks, manual API run	Internal team / beta	Working Orchestrator → Executor → Validator loop
Alpha (Phase 3–4)	PM Agent + Architect Agent, RAG, multi-file tasks, GitHub	Early adopters (100 users)	50% task success rate from PRD input
Beta (Phase 5)	Full IDE, QA Agent, DevOps Agent, job history, approvals	Paying users (1K users)	80% task success + deployed app from PRD
V1.0 (Phase 6)	Kubernetes, billing, multi-tenant, advanced AI routing	SMB engineering teams	1K daily active jobs
V2.0 (Future)	Multi-agent parallel execution, custom agent training, plugin API	Enterprise customers	10K daily active jobs
V3.0 (Vision)	Autonomous full-stack delivery from spec alone	Platform / API users	End-to-end autonomy with human-in-loop checkpoints

26. Final Conclusion

This blueprint defines a complete, production-grade AI Software Development Agency built on a Python-first stack with 11 specialized AI agents. Every connection between agents is defined, every handoff protocol is specified, and every layer — from LLM integration to sandbox security to observability — has been designed with real-world constraints in mind.

Golden Rules

- Working MVP first. Ship a functional Orchestrator → Executor → Validator loop before building the full crew.
- Python-first. Use FastAPI, asyncio, and Celery consistently throughout. No Node.js backend.
- Agents never call each other directly. All communication flows through the Redis message bus.
- Stateless agents. All state lives in Redis (ephemeral) and PostgreSQL (persistent).
- Security by design. Sandbox isolation and prompt injection defense are built-in from Phase 1.
- Fail gracefully. Every agent degrades cleanly — partial results beat silent failures.
- Cost-aware architecture. Token budgets and model routing are mandatory, not optional.

Critical Success Factors

Pillar	What It Means	Key Metric
AI Quality	Agents produce correct, style-consistent, well-tested code	Task success rate > 80%
Agent Connectivity	Zero dropped messages between agents; full audit trail	0 unhandled agent failures
Security	No sandbox escapes, no data leakage between tenants	Zero critical CVEs
Reliability	Jobs complete without human intervention	Job failure rate < 5%
Performance	Users get results within acceptable time	P95 job latency < 3 minutes
Cost Efficiency	LLM costs scale sub-linearly with usage	< $2 average cost per job (existing repos)

> Living Document: Update this blueprint as the system evolves. Every architectural decision, agent addition, trade-off, and lesson learned should be captured here for the team.